

Network Traffic Analysis using Wireshark

Introduction

This project was conducted in a controlled laboratory environment using Burp Suite and DVWA (Damn Vulnerable Web Application) to understand the fundamentals of web application security testing. The focus of the project was to intercept, inspect, analyze, and modify HTTP or HTTPS traffic between a web browser and a web application while observing how authentication, sessions, and user requests are handled.

Objective

1. Learn the working of HTTP and HTTPS protocols.
2. Configure Burp Suite as an intercepting proxy.
3. Capture and analyze HTTP requests and responses.
4. Examine request methods, headers, cookies, and parameters.

Lab Environment

1. Operating System - Kali Linux
2. Proxy tools - Burp Suite Professional Edition
3. Vulnerable application - DVWA (Damn Vulnerable Web Application)
4. Web Server - Apache
5. Database - MariaDB
6. Browser - Firefox
7. Proxy - Foxy Proxy

Methodology

1. Environment Setup

1. The following components were installed and Configured.

MySQL Database

DVWA

Foxy Proxy

Burp Suite Professional Edition

```
(parasyte@kali)-[/var/www/html]
$ sudo git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5746, done.
remote: Total 5746 (delta 0), reused 0 (delta 0), pack-reused 5746 (from 1)
Receiving objects: 100% (5746/5746), 2.76 MiB | 8.40 MiB/s, done.
Resolving deltas: 100% (2857/2857), done.
```

```
GNU nano 8.7.1 config.inc.php
## WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
# See README.md for more information on this.
$_DVWA = array();
$_DVWA['db_server'] = getenv('DB_SERVER') ? '127.0.0.1';
$_DVWA['db_database'] = getenv('DB_DATABASE') ? 'dvwa';
$_DVWA['db_user'] = getenv('DB_USER') ? 'dvwa';
$_DVWA['db_password'] = getenv('DB_PASSWORD') ? 'p@ssw0rd';
$_DVWA['db_port'] = getenv('DB_PORT') ? '3306';

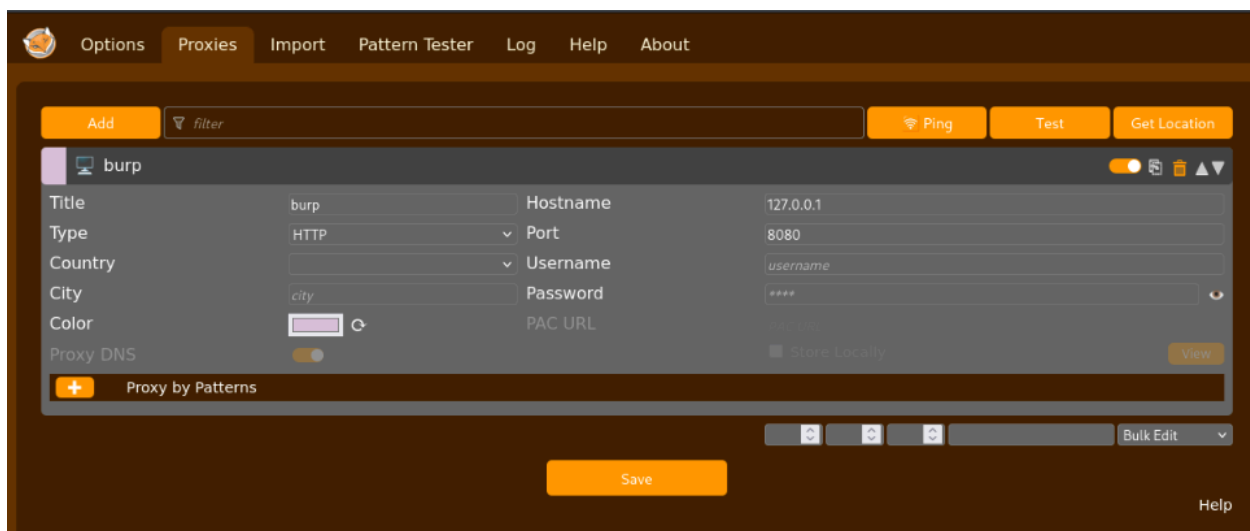
# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$_DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ? '';
$_DVWA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ? '';

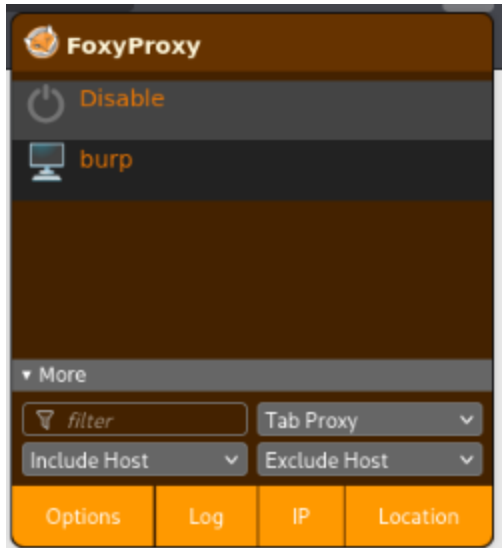
# Default security level
# Default value for the security level with each session.
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or impossible'.
$_DVWA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ? 'impossible';
```

```
MariaDB [(none)]> CREATE DATABASE dvwa;  
Query OK, 1 row affected (0.002 sec)  
  
MariaDB [(none)]>  
MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'password';  
Query OK, 0 rows affected (0.010 sec)  
  
MariaDB [(none)]>  
MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';  
Query OK, 0 rows affected (0.003 sec)  
  
MariaDB [(none)]>  
MariaDB [(none)]> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.001 sec)  
  
MariaDB [(none)]>  
MariaDB [(none)]> EXIT;  
Bye
```

2. Proxy Configuration

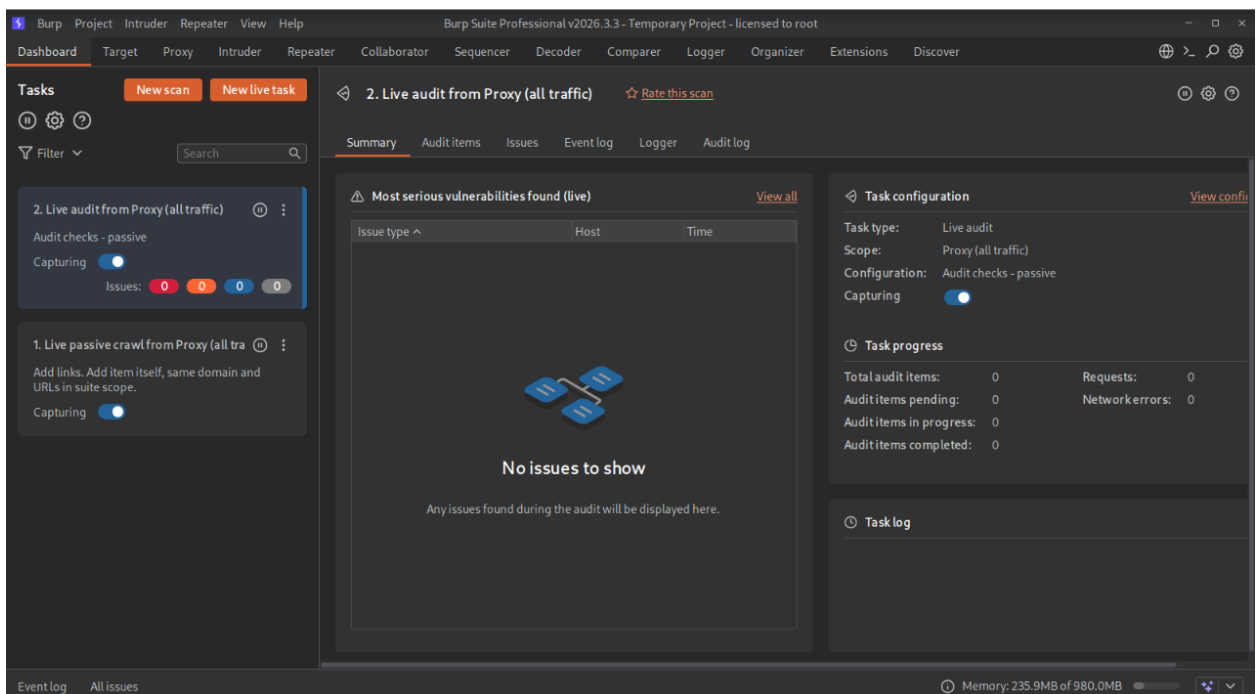
1. Proxy - Foxy
2. HTTP Proxy - 127.0.0.1
3. Port - 8080
4. Title - Burp



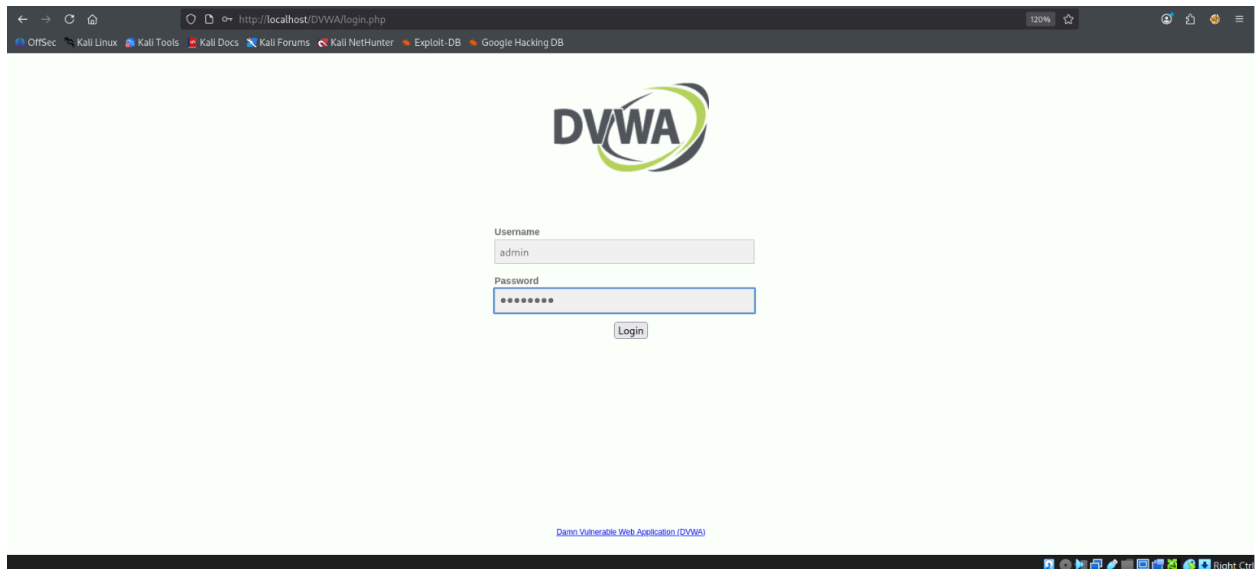


3. Switched on Burpsuite

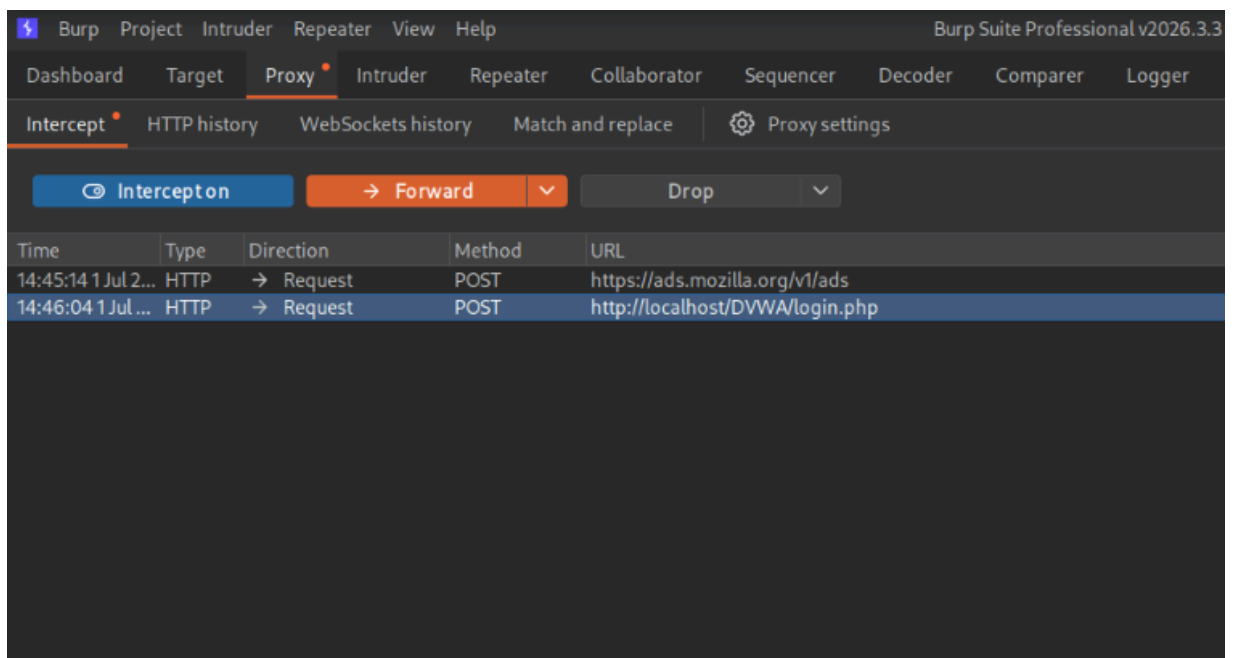
1. After proxy switch on burp to analyse the http/https request.
2. Start the Intercept to stop requests on the dashboard.



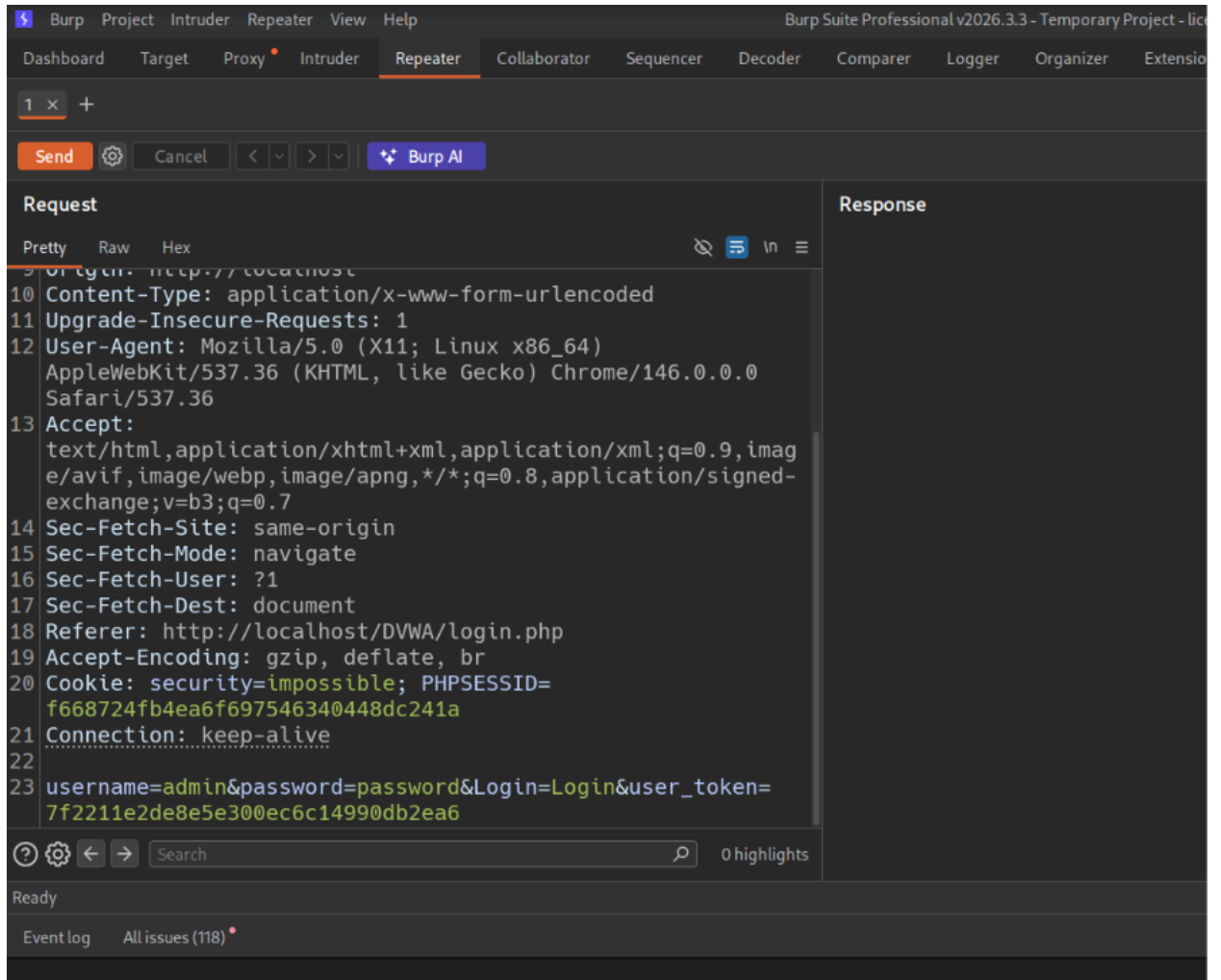
4. Login in Into DVWA (Closed Environment)



5. Intercept Request



6. Send to Repeater



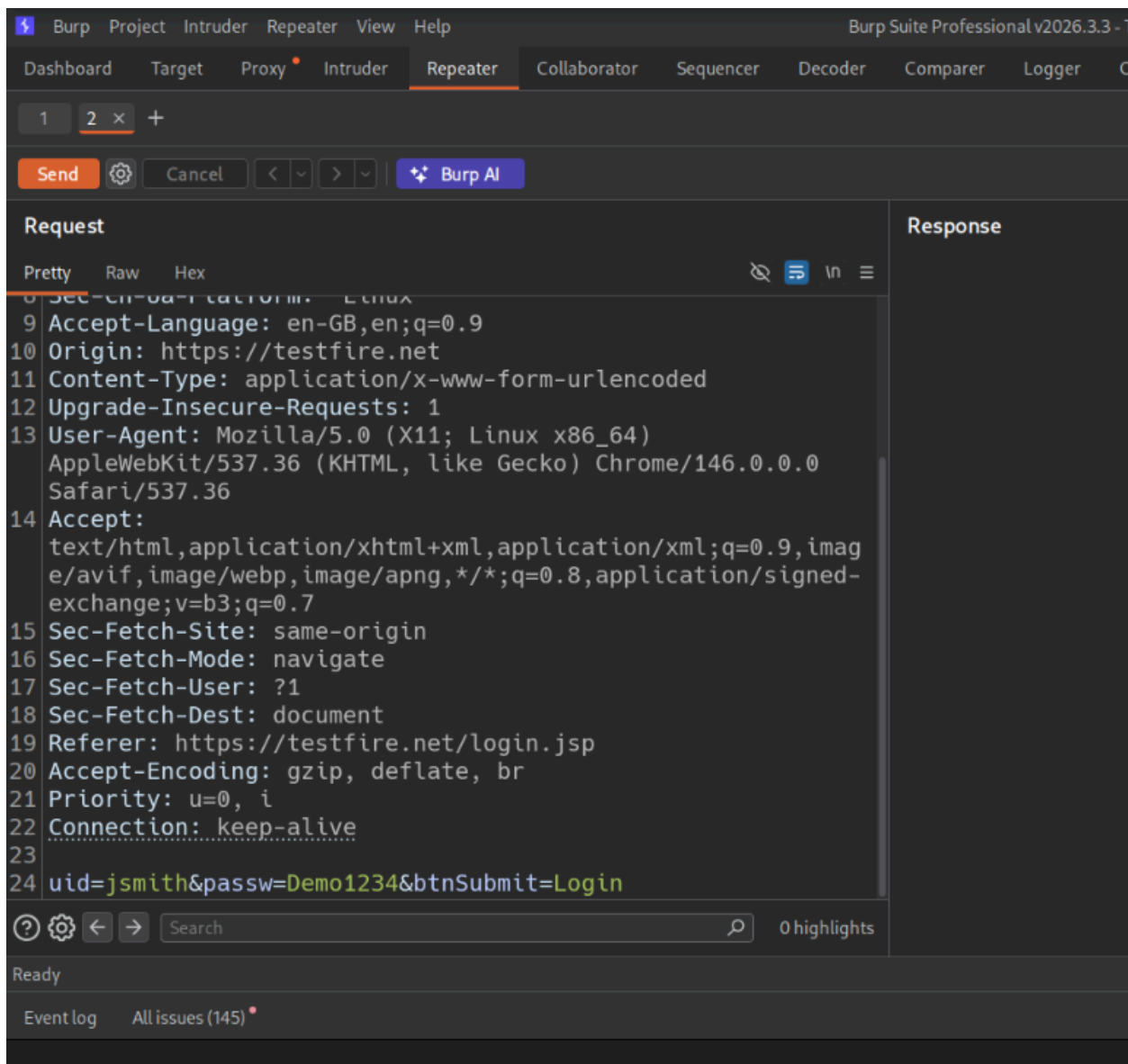
Username = admin

Password = password

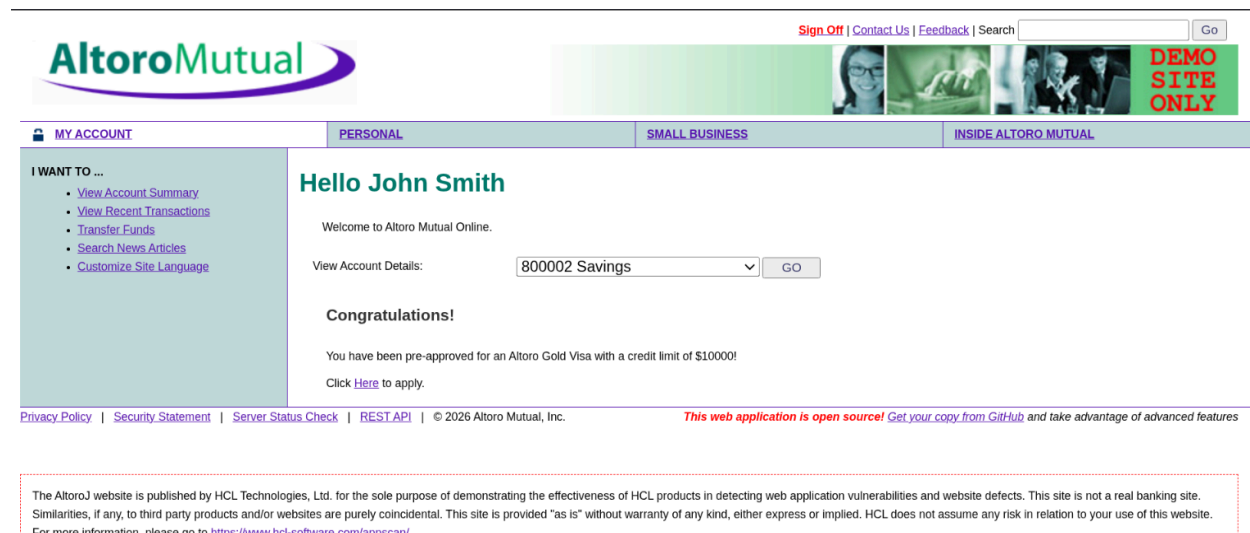
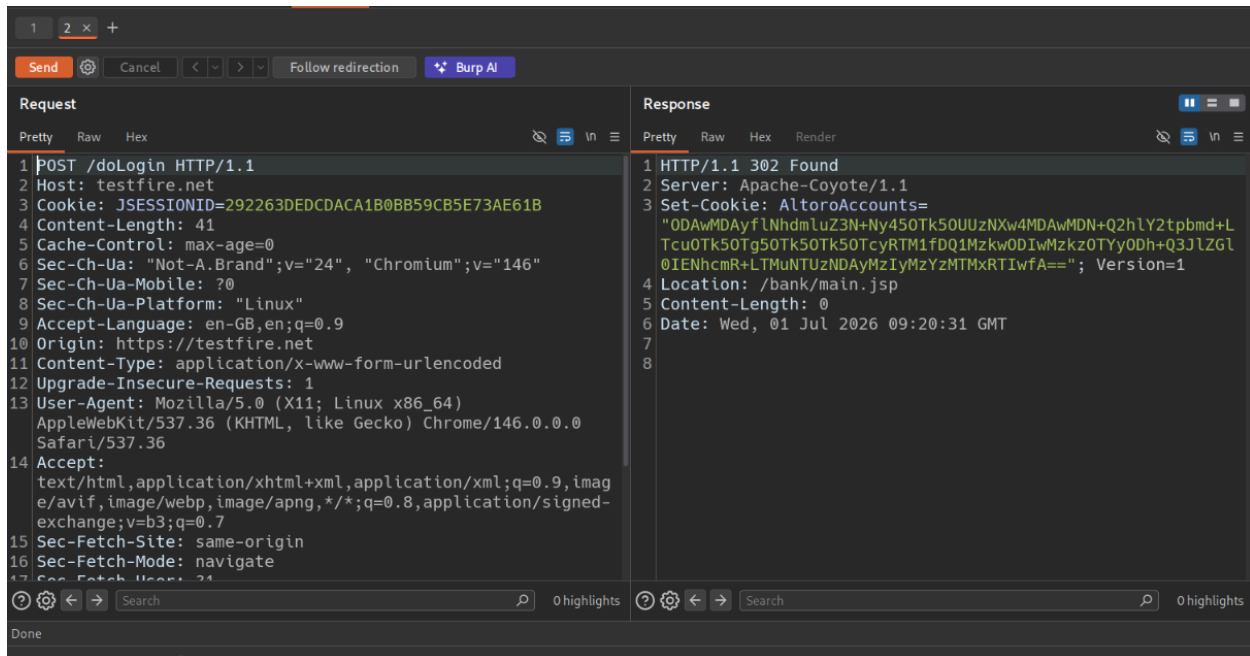
- **Web Application Security Testing Using Burp Suite and Altoro Mutual**

Methodology

1. Login on the Altoro Mutual and catch the request in BurpSuite.



2. Forward the request from repeater



Conclusion

This project provided practical experience in web application security testing using Burp Suite with DVWA and Altoro Mutual in a controlled environment. By intercepting HTTP/HTTPS traffic, inspecting requests, responses, cookies, and sessions, and I gained a better understanding of web application communication, authentication, and security testing

methodologies. Overall, the project strengthened my foundational skills in web application security, traffic analysis, and vulnerability assessment while emphasizing the importance of ethical and authorized security testing.